



CHRIST

(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

OBSYS: An Interactive Framework for Operating System Simulation via GUI and Terminal Interfaces

by

Arnav Narula (2547115)

Nayana K Benny (2547136)

Shambhavi Sinha (2547151)

Under the guidance of

Dr. Neha Singhal

A Project report submitted in partial fulfilment of the requirements of

III Trimester Master of Computer Applications,

CHRIST (Deemed to be University)

March 2026



CHRIST

(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

CERTIFICATE

*This is to certify that the report titled **OBSYS: An Interactive Framework for Operating System Simulation via GUI and Terminal Interfaces** is a bonafide record of work done by Arnav Narula (2547115), Nayana K Benny (2547136) and Shambhavi Sinha (2547151) of CHRIST (Deemed to be University), Bangalore, in partial fulfilment of the requirements of III Trimester MCA during the year 2025-2026.*

Head of the Department

cc - 17/5/2026
Project Guide

Valued-by:

1. Dr. Ashwini Patil
2. *Dr. K*
20/3/26

Name : Shambhavi Sinha
Register Number : 2547151
Examination Centre : CHRIST (Deemed to be University)
Date of Exam :

AI Usage Declaration Statement

I hereby declare that Artificial Intelligence tools were used only for supportive purposes such as idea exploration, conceptual understanding, literature guidance, and grammar refinement during the development of this project. No AI generated code has been copied or submitted, and all programming, model development, experimentation, analysis, and results were independently developed and implemented by me or our team. I confirm that this project is our original work and reflects our own understanding and effort, and I take full responsibility for its authenticity and adherence to academic integrity policies.

I also declare that the approximate usage of Artificial Intelligence tools for supportive purposes in this project is 30%, limited to idea exploration, literature search guidance, grammar refinement, clarification of technical concepts, and other academic assistance.

The details of AI tools used in this project and their purpose are listed below:

1. **Name of the AI Tool:** Claude
Purpose of Use: Idea exploration and Tech stack recommendation
2. **Name of the AI Tool:** Perplexity
Purpose of Use: Literature Review
3. **Name of the AI Tool:** Gemini
Purpose of Use: Code Generation and Logic Simplification

This declaration confirms that the AI tools mentioned above were used only for supportive and assistive purposes and that no AI generated code has been directly copied or submitted as part of this project.

Name of the Student : Shambhavi Sinha

Register Number : 2547151

Project Title : OBSYS: An Interactive Framework for Operating System

Simulation via GUI and Terminal Interfaces

Signature : *Shambhavi*

Date : 17/03/2026

ABSTRACT

With the modernization of operating systems, it has become significantly simpler for systems to perform complex process management along with efficient scheduling operations. These operations are mostly abstracted from the users and developers, making it difficult for them to understand the overall flow of how a task is executed at the system level. Traditional methods of teaching operating system concepts rely heavily on static textbook illustrations, theoretical explanations, and simplified pseudocode examples. While these approaches establish foundational understanding, they fall short of conveying the dynamic, time-dependent nature of scheduling decisions and the cascading effects they have on system performance. Students and developers are often left without an intuitive sense of how processes compete for CPU time, transition between states, or how different policies alter overall system efficiency.

OBSYS addresses this gap by serving as an interactive framework for operating system simulation via GUI and terminal interfaces. OBSYS integrates an external command-line interface (CLI) and a Graphical User Interface (GUI) to provide a comprehensive understanding of the entire system flow of task execution and coordination. The CLI serves as an input medium that mirrors the low-level, operating-system-like interaction model familiar to developers, while the GUI provides a platform for real-time visualization of process queues, CPU timelines, and scheduling decisions as they unfold. The backend implements four CPU scheduling algorithms — First Come First Serve (FCFS), Round Robin (RR), Priority Scheduling, and Shortest Job First (SJF) — driven by a discrete, clock-based simulation engine that accurately models process arrivals, state transitions, queue movements, and CPU execution. A REST API and WebSocket communication layer expose these functionalities to both interfaces simultaneously, ensuring that the CLI and GUI remain synchronized throughout every tick of the simulation.

TABLE OF CONTENTS

Acknowledgments

List of Figures

1. Introduction
 - 1.1. Project Description
 - 1.2. Existing System
 - 1.3. Objectives
 - 1.4. Purpose, Scope and Applicability
 - 1.4.1. Purpose
 - 1.4.2. Scope
 - 1.4.3. Applicability
 - 1.5. Overview of the Report
2. System Analysis and Requirements
 - 2.1. Problem Definition
 - 2.2. Requirements Specification
 - 2.3. Block Diagram
 - 2.4. System Requirements
 - 2.4.1. User Characteristics
 - 2.4.2. Software and Hardware Requirements
 - 2.4.3. Constraints
 - 2.5. Conceptual Models
 - 2.5.1. Data Flow Diagram
 - 2.5.2. DB Design
3. System Design
 - 3.1. System Architecture
 - 3.2. Module Design

- 3.3. Interface Design and Procedural Design**
 - 3.3.1. User Interface Design**
 - 3.3.2. Class Diagram**
- 4. Implementation**
 - 4.1. Implementation Approaches**
 - 4.2. Coding Standard**
 - 4.3. Coding Details**
 - 4.4. Screen Shots**
- 5. Testing**
 - 5.1. Test Cases**
 - 5.2. Testing Approaches**
 - 5.3. Test Reports**
- 6. Conclusion**
 - 6.1. Design and Implementation Issues**
 - 6.2. Advantages and Limitations**
 - 6.3. Future Scope of the Project**

ACKNOWLEDGEMENT

First of all, I thank the Lord almighty for his immense grace and blessings showered on me at every stage of this work.

I profoundly thank Dr. Fr. Jossy P George, Director, Department of Computer Science, CHRIST (Deemed to be University), Bangalore, for providing me the opportunity to be a part of the institution.

I am greatly indebted to the Head of the Department of Computer Science, Dr. Rupali Sunil Wagh, CHRIST (Deemed to be University) for providing the opportunity to take up this project as part of my curriculum.

I am greatly indebted to our Software Project Coordinator, Dr. Sridevi R, for providing the opportunity to take up this project and for helping with her valuable suggestions.

I am deeply indebted to our project guide Dr. Neha Singhal, for her assistance and valuable suggestions as a guide. Their critical suggestions helped us to improve the project work.

I express my sincere thanks to all Faculties of the Department of Computer Science, CHRIST (Deemed to be University), for their valuable suggestions during the course of this project.

Acknowledging the efforts of everyone, their chivalrous help in the course of the project preparation and their willingness to corroborate with the work, their magnanimity through lucid technical details lead to the successful completion of my project.

I would like to express my sincere thanks to all my friends, colleagues, parents and all

LIST OF FIGURES

Figure	Title	Page Number
2.1	OBSYS Block Diagram	9
2.2	OBSYS Data Flow Diagram	12
2.3	OBSYS DB Design	13
3.1	OBSYS Module Design	16
3.2	OBSYS Class Design	23

Chapter 1: Introduction

Modern operating systems perform complex process management and scheduling operations that often remain invisible to learners and developers, making it difficult to understand how tasks are executed, prioritized, and coordinated at the system level. This project, titled OBSYS : An Interactive Framework for Operating System Simulation via GUI and Terminal Interfaces, presents an interactive platform that simulates operating system scheduling behavior and provides both a command-line interface (CLI) and a graphical user interface (GUI) for real-time visualization.

1.1. Project Description

Modern operating systems have evolved into highly sophisticated environments capable of managing hundreds of concurrent processes through complex scheduling mechanisms, memory allocation strategies, and resource coordination protocols. While this evolution has made systems more powerful and efficient, it has simultaneously created a layer of abstraction that renders the inner workings of the OS invisible to those who need to understand them most, students, early-career developers, and researchers. The dynamic behavior of a scheduler, including how it decides which process runs next, how long it runs, and what happens to waiting processes in the meantime, is rarely observable in a real system, making it one of the most conceptually challenging topics in computer science education. Existing approaches to teaching these concepts, including static textbook diagrams, paper-based exercises, and isolated code walkthroughs, fail to convey the time-dependent, state-driven nature of scheduling decisions and their measurable impact on system performance.

OBSYS (Operating System Behavior Simulator) is designed to directly address this gap. It is an interactive simulation framework that replicates the core CPU scheduling behavior of an operating system through a dual-interface architecture comprising an external Command-Line Interface (CLI) for structured user interaction and a web-based Graphical User Interface (GUI) for real-time visualization. The backend of OBSYS is

built around a discrete, clock-driven simulation engine that models the full lifecycle of a process: from arrival and admission into the ready queue, through CPU execution and preemption, to final completion. Four scheduling algorithms are implemented within this engine, namely First-Come-First-Serve (FCFS), Round Robin (RR), Priority Scheduling, and Shortest Job First (SJF), each of which can be selected and switched interactively during a simulation session. The engine exposes its functionality through a REST API and WebSocket communication layer built using FastAPI and Uvicorn, which enables continuous, low-latency streaming of live system state to all connected clients.

The CLI component, implemented in Python using the Typer library, allows users to define processes, configure scheduling policies, and control simulations entirely from the terminal, mirroring the low-level interaction model of a real operating system. The GUI dashboard, developed in TypeScript and React, subscribes to real-time backend updates via WebSocket and renders animated Gantt charts, process queue tables, CPU execution timelines, and performance metric panels including average waiting time, turnaround time, CPU utilization, and throughput, all of which update live at every simulation tick. Both interfaces communicate simultaneously with the same backend, ensuring their views remain fully synchronized throughout the simulation lifecycle. A key strength of OBSYS is its cross-platform design: the system runs identically on Windows, Linux, and macOS, and the core simulation logic is fully decoupled from both interfaces, making the framework extensible and suitable for deployment in diverse educational and research environments.

1.2. Existing Systems

Operating systems have evolved into highly sophisticated environments capable of managing hundreds of concurrent processes through complex scheduling mechanisms and resource coordination protocols. This evolution, while making systems more powerful, has created a deep layer of abstraction that renders the inner workings of the OS invisible to students and developers. The dynamic behavior of a scheduler, how it

decides which process runs next, how long it runs, and what happens to waiting processes is rarely observable in a real system, making it one of the most challenging topics in computer science education. Conventional approaches such as static textbook diagrams and paper-based exercises fail to convey the time-dependent, state-driven nature of scheduling decisions and their measurable impact on system performance.

Several simulation tools currently exist that attempt to address this. GraySim offers dynamic feedback for practicing CPU scheduling policies, while tools like the UR-OS simulator allow users to create and evaluate scheduling algorithms in a Java-based environment. Browser-based graphical simulators and virtual simulation tools provide visual demonstrations of scheduling and memory management for students. However, these tools share common limitations: most are platform-specific or browser-dependent, none integrate a real external CLI alongside a GUI, and real-time state streaming is largely absent across all of them.

OBSYS addresses these limitations by providing an interactive simulation framework built around a discrete, clock-driven engine that implements four scheduling algorithms: FCFS, Round Robin, Priority Scheduling, and SJF. The engine is exposed through a REST API and WebSocket layer built using FastAPI and Uvicorn, enabling live state streaming to both a Python-based CLI tool and a React-based GUI dashboard simultaneously. Users can define processes, select algorithms, and observe real-time Gantt charts, queue transitions, and performance metrics including waiting time, turnaround time, CPU utilization, and throughput across all supported platforms: Windows, Linux, and macOS.

1.3. Objectives

The objectives of OBSYS are as follows:

- To design and implement a discrete, clock-driven simulation engine that models process lifecycle, state transitions, and queue management across four scheduling algorithms: FCFS, Round Robin, Priority Scheduling, and Shortest Job First.
- To expose the simulation engine as a backend service using REST API and WebSocket communication, enabling continuous real-time streaming of live system state to connected clients.
- To develop an external CLI tool and a web-based GUI dashboard that allow users to define processes, configure scheduling policies, and observe simulation behavior concurrently through complementary interfaces.
- To ensure cross-platform compatibility across Windows, Linux, and macOS while maintaining a fully decoupled architecture where the core simulation logic remains independent of any specific interface.
- To serve as an educational tool that enables students, instructors, and system enthusiasts to experiment with scheduling algorithms and observe their impact on performance metrics such as waiting time, turnaround time, CPU utilization, and throughput.

1.4. Purpose, Scope and Applicability

1.4.1. Purpose

The purpose of OBSYS is to bridge the gap between theoretical knowledge of operating system scheduling and practical, observable understanding. Operating system internals are largely invisible during normal system usage, leaving students and developers with little more than abstract descriptions of how scheduling works. OBSYS exists to change this by providing a controlled, interactive environment where scheduling behavior can be triggered, observed, and analyzed in real time. The significance of this project lies in its ability to make a fundamentally abstract topic tangible without requiring access to or modification of an actual OS kernel. By combining a CLI and a GUI over a shared simulation backend, OBSYS improves upon existing tools by offering an authentic, dual-mode experience that reflects how real systems are interacted with and monitored.

1.4.2. Scope

OBSYS covers the simulation of CPU scheduling algorithms, process state management, ready queue evolution, and performance metric computation. The system accepts user-defined processes with attributes such as arrival time, burst time, and priority, and simulates their execution under the selected scheduling policy in discrete time steps. The scope includes four algorithms: FCFS, Round Robin, Priority Scheduling, and SJF. The system does not simulate memory management, I/O scheduling, or inter-process communication, as these fall outside the defined boundaries of this project. The simulation is clock-driven and does not replicate real-time kernel-level execution. Key assumptions include that all processes are provided by the user prior to or during simulation, and that the environment has Python 3 and a modern web browser available for the backend and GUI respectively.

1.4.3. Applicability

OBSYS is primarily applicable as an educational tool for undergraduate and postgraduate students studying operating systems, where it can be used in classroom demonstrations, laboratory sessions, and self-directed learning. Instructors can use it to visually demonstrate the behavioral differences between scheduling algorithms during lectures. Beyond academia, OBSYS is applicable in research contexts where scheduling algorithm performance needs to be compared and analyzed under controlled conditions. Indirectly, the project contributes to society by lowering the barrier to understanding critical system-level concepts, producing graduates and developers who have a clearer, more practical understanding of how operating systems manage computational resources.

1.5. Overview of the Report

This report is organized into six chapters. Chapter 2 presents a system analysis covering the problem definition, requirements specification, and conceptual models including data flow diagrams and an entity-relationship diagram. Chapter 3 describes the system design, detailing the architecture, module breakdown, database design, and interface design of OBSYS. Chapter 4 covers the implementation, discussing the development approach, coding standards, and key components of the system alongside relevant screenshots. Chapter 5 presents the testing strategy, defining test cases, testing approaches, and test results. Chapter 6 concludes the report with a discussion of design and implementation challenges, advantages and limitations of the system, and directions for future development.

Chapter 1: Introduction

Chapter 2: System Analysis and Requirements

Chapter 3: System Design

Chapter 4: Implementation

Chapter 5: Testing

Chapter 6: Conclusion

Chapter 2: System Analysis and Requirements

2.1. Problem Definition

The core problem addressed by OBSYS is the opacity of CPU scheduling operations in modern operating systems. Students and developers studying operating systems are unable to directly observe how processes compete for CPU time, transition between execution states, or how different scheduling policies affect overall system performance without either modifying a live OS kernel or relying on static textbook illustrations. This disconnect between theory and observable behavior makes scheduling one of the most difficult concepts to internalize in computer science education.

The problem can be broken down into the following sub-problems. First, there is no widely available tool that provides both a CLI and a GUI over the same simulation backend, leaving learners with either a purely visual experience or a purely terminal-based one, but never both simultaneously. Second, existing simulators are largely platform-specific or browser-dependent, limiting their accessibility across different environments. Third, most tools present scheduling results as static, pre-computed outputs rather than streaming live state updates, which fails to capture the dynamic, evolving nature of a real scheduler. Fourth, no existing tool provides a clean, decoupled architecture that allows the simulation engine to be accessed and extended independently of any interface.

2.2. Requirements Specifications

The requirements of OBSYS are derived from the limitations identified in the problem definition and are divided into functional and non-functional requirements.

Functional Requirements:

- The system shall allow users to create processes by specifying attributes including process ID, arrival time, burst time, and priority.
- The system shall support four scheduling algorithms: FCFS, Round Robin, Priority Scheduling, and Shortest Job First.

- The system shall simulate scheduling in discrete time steps and emit live state updates at each clock tick.
- The system shall provide a CLI tool through which users can add processes, select algorithms, and control the simulation via structured terminal commands.
- The system shall provide a web-based GUI dashboard that displays animated Gantt charts, process queue tables, and performance metrics updated in real time.
- The system shall allow users to start, pause, reset, and switch between scheduling algorithms interactively during a session.
- The system shall compute and display performance metrics including average waiting time, turnaround time, CPU utilization, and throughput upon simulation completion.

Non Functional Requirements:

- The system shall be cross-platform, running consistently on Windows, Linux, and macOS.
- The CLI and GUI shall remain synchronized through real-time WebSocket communication with the shared backend.
- The system shall be modular and extensible, allowing new scheduling algorithms to be added without modifying existing components.
- The simulation engine shall remain fully decoupled from both interfaces, functioning as an independent, reusable service.
- UI updates shall reflect simulation state changes with minimal perceptible latency during normal operation.

2.3. Block Diagram

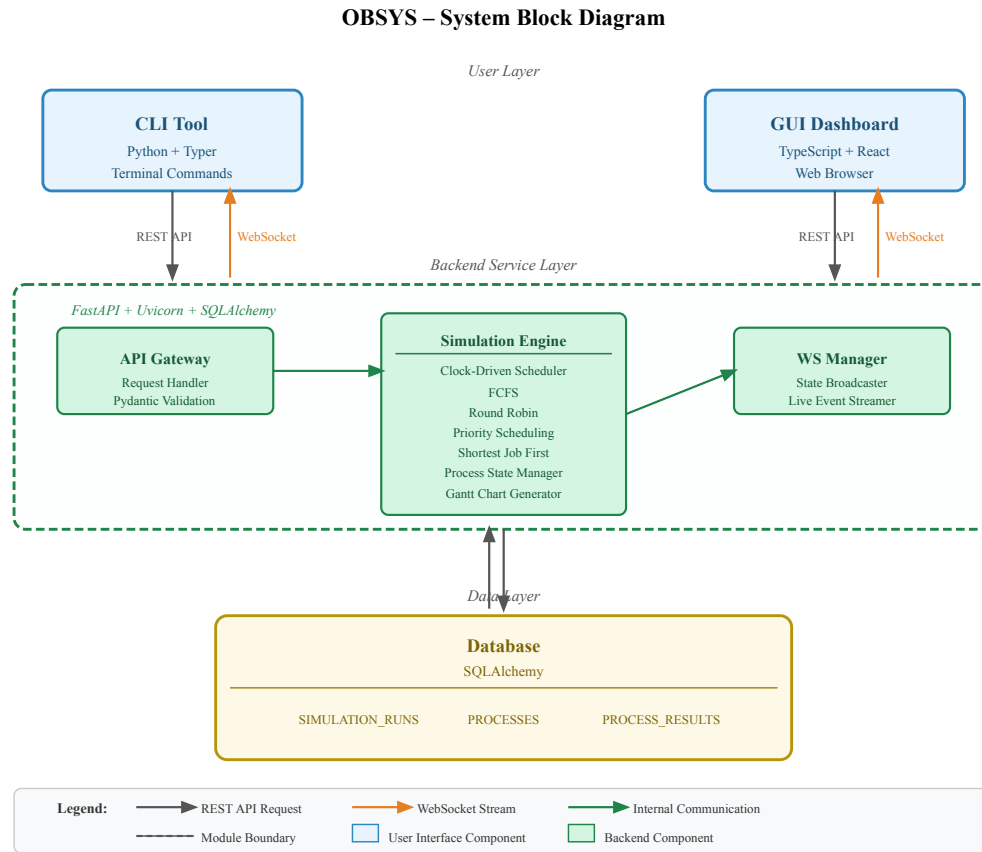


Fig. 2.1. OBSYS Block Diagram

- **User Layer** — the CLI Tool and GUI Dashboard sitting side by side, each communicating with the backend via REST API (for requests) and WebSocket (for live updates)
- **Backend Service Layer** — the API Gateway, Simulation Engine (with all four algorithms listed), and WebSocket Manager, all enclosed within the FastAPI + Uvicorn boundary
- **Data Layer** — the database with three tables

2.4. System Requirements

2.4.1. User Characteristics

OBSYS is designed to serve three primary categories of users. The first category is students, particularly undergraduate and postgraduate learners enrolled in operating systems courses, who use OBSYS to observe and understand CPU scheduling behavior in an interactive and visual manner. These users are expected to have a basic familiarity with OS concepts but are not required to have any programming experience to operate the GUI. The second category is instructors and educators who use OBSYS as a classroom or laboratory demonstration tool to visually illustrate the differences between scheduling algorithms during lectures. The third category is system enthusiasts and researchers who wish to experiment with scheduling policies, compare algorithm performance under different process configurations, and use OBSYS as an analytical platform for studying scheduling behavior in a controlled environment.

2.4.2. Software and Hardware Requirements

Software Requirements:

- **Operating System:** Windows 10 or later, Ubuntu 20.04 or later, or macOS 12 or later
- **Backend:** Python 3.10 or later, FastAPI, Uvicorn, Pydantic, asyncio, WebSockets, SQLAlchemy
- **CLI Tool:** Python 3.10 or later, Typer, asyncio, requests
- **Frontend:** Node.js 18 or later, TypeScript, React 18 or later, Tailwind CSS, Recharts, Framer Motion
- **Development Tools:** Visual Studio Code, PyInstaller for CLI packaging, Docker (optional)
- **Browser:** Any modern web browser with WebSocket support such as Chrome, Firefox, or Edge

Hardware Requirements:

- **Processor:** Intel Core i3 or equivalent, minimum
- **RAM:** 4 GB minimum, 8 GB recommended
- **Disk Space:** 500 MB for dependencies and project files
- **Display:** 1280 x 720 resolution minimum for the GUI dashboard

2.4.3. Constraints

The simulation is discrete and clock-driven and does not replicate actual kernel-level OS execution behavior, meaning results reflect a model of scheduling rather than a live system. Memory management, I/O scheduling, and inter-process communication are outside the scope of this simulation and are not modeled. The GUI requires a modern web browser with WebSocket support enabled and is not designed for screen resolutions below 1280 x 720 pixels. The current version of OBSYS supports a single active simulation session per backend instance and does not handle concurrent multi-user sessions. All process attributes including arrival time, burst time, and priority must be supplied by the user, as the system does not generate or infer process workloads automatically.

2.5. Conceptual Models

To better understand and represent the structure and behavior of OBSYS, a set of conceptual models has been developed. These models capture the functional flow of data through the system, the relationships between core entities, and the sequence of interactions between users, interfaces, and the simulation backend. The models presented in this section include a Data Flow Diagram, which illustrates how inputs are processed and outputs are generated across the system, and an Entity Relationship Diagram, which defines the structure of the database and the associations between stored data entities. Together, these models serve as the blueprint that guided the design and implementation of OBSYS.

2.5.1. Data Flow Diagram

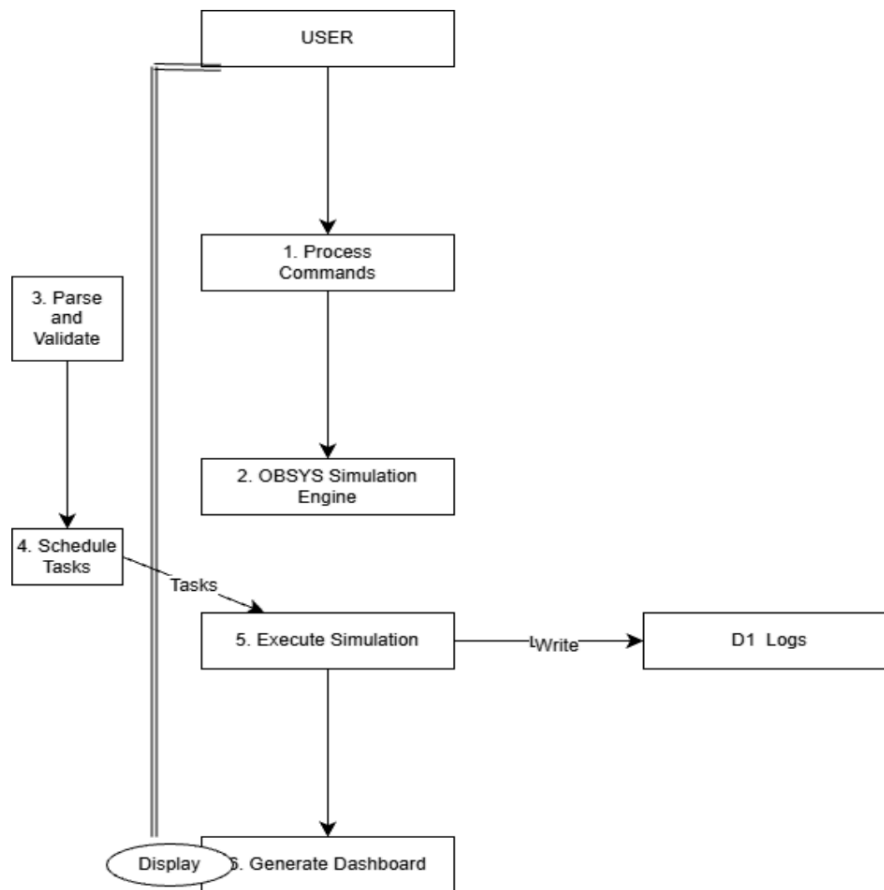


Fig. 2.2 OBSYS Data Flow Diagram

Figure 2.2 illustrates the data flow diagram of OBSYS. The user initiates the system by submitting commands, which are first received and processed by the command handler. The input is then parsed and validated before being passed to the scheduling module, which organizes the tasks accordingly. The scheduled tasks are forwarded to the OBSYS Simulation Engine, which executes the simulation and simultaneously writes execution logs to the data store. Once the simulation is complete, the results are sent to the dashboard generation module, which produces the final visual output displayed to the user.

2.5.2 DB Design

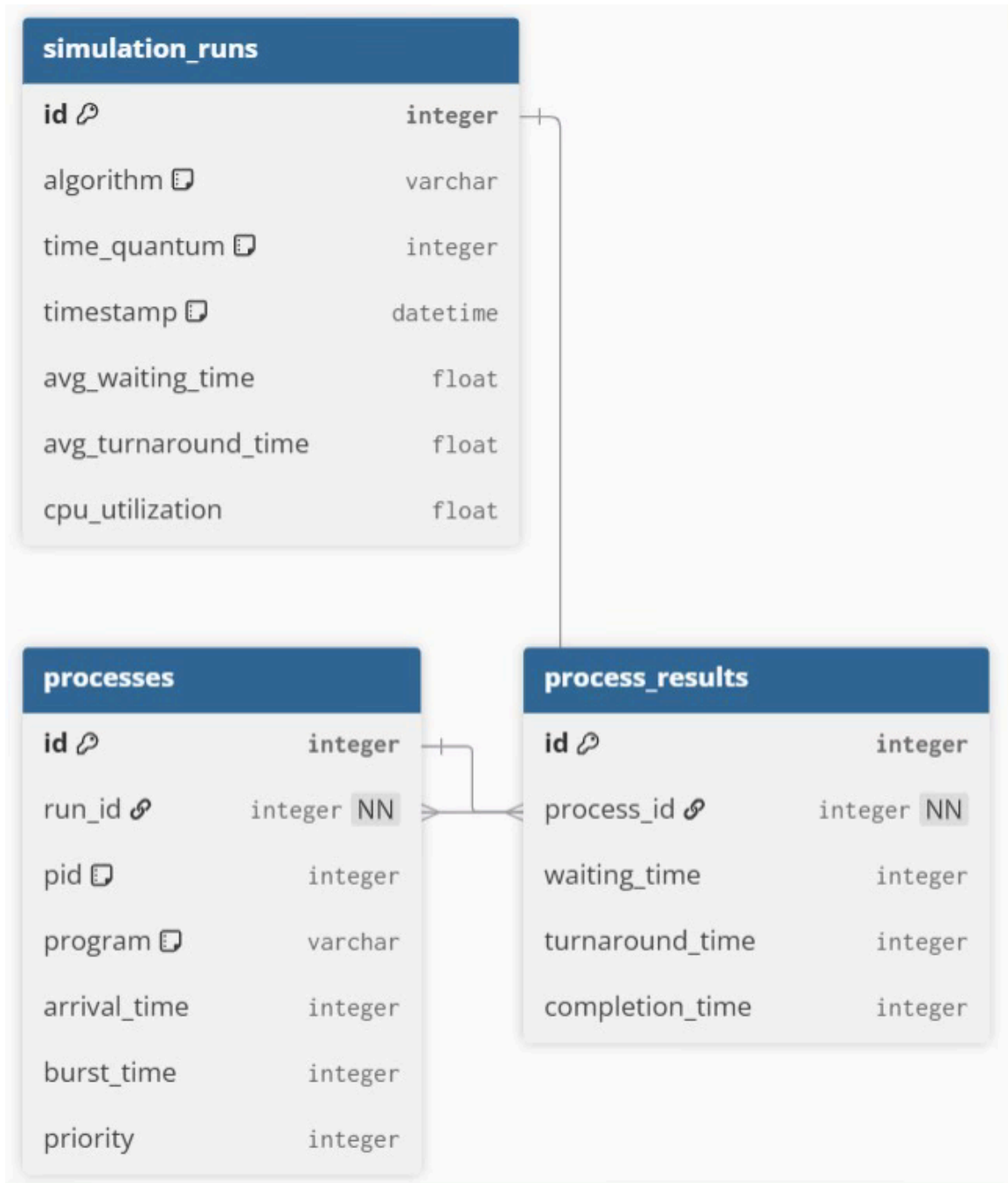
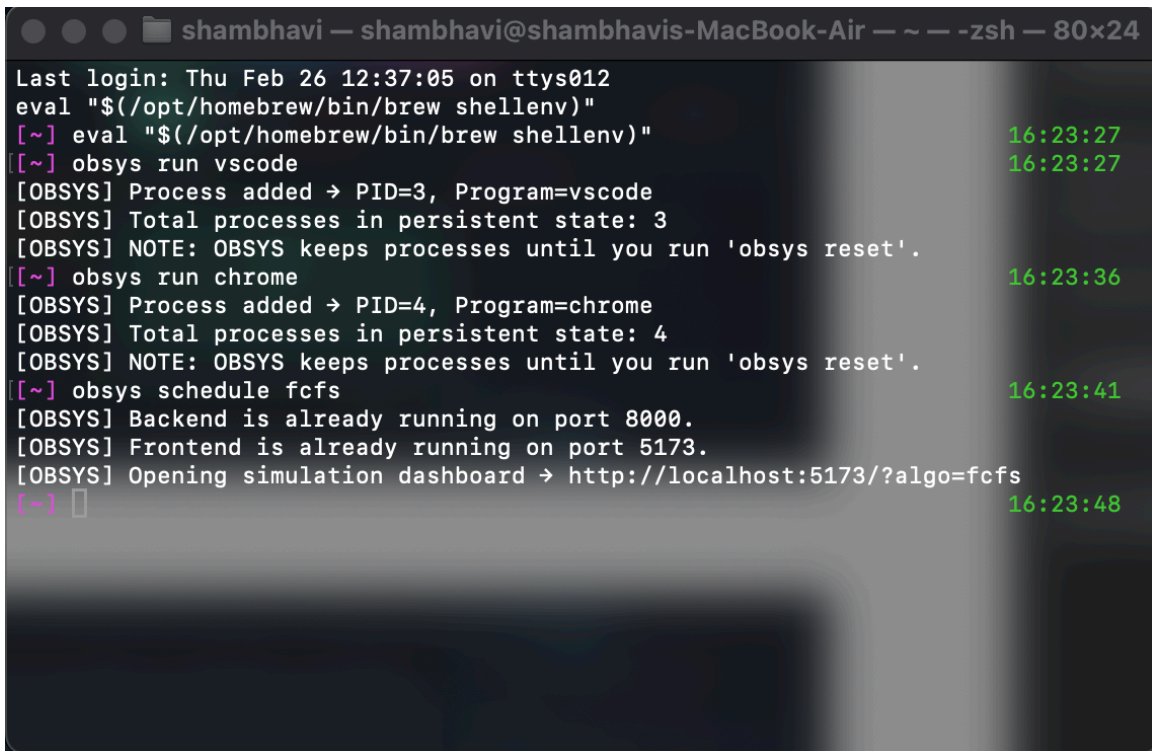


Fig. 2.3 OBSYS DB Design

A terminal window titled 'shambhavi — shambhavi@shambhavis-MacBook-Air — ~ — -zsh — 80x24'. The terminal shows the following commands and output:

```
Last login: Thu Feb 26 12:37:05 on ttys012
eval "$(/opt/homebrew/bin/brew shellenv)"
[~] eval "$(/opt/homebrew/bin/brew shellenv)"
[~] obsys run vscode
[OBSYS] Process added → PID=3, Program=vscode
[OBSYS] Total processes in persistent state: 3
[OBSYS] NOTE: OBSYS keeps processes until you run 'obsys reset'.
[~] obsys run chrome
[OBSYS] Process added → PID=4, Program=chrome
[OBSYS] Total processes in persistent state: 4
[OBSYS] NOTE: OBSYS keeps processes until you run 'obsys reset'.
[~] obsys schedule fcfs
[OBSYS] Backend is already running on port 8000.
[OBSYS] Frontend is already running on port 5173.
[OBSYS] Opening simulation dashboard → http://localhost:5173/?algo=fcfs
[~]
```

3.1 System Architecture

The architecture of OBSYS follows a layered, service-oriented design where the core simulation logic is fully decoupled from the user-facing interfaces. The system is composed of five principal modules, each with clearly defined responsibilities.

- **Command Line Interface Module:**

The Command Line Interface module is built using Python and Typer and serves as the terminal-based interaction layer of OBSYS. It allows users to issue structured commands for process creation, algorithm selection, and simulation control without requiring access to a graphical environment. This module communicates with the backend by sending HTTP requests to the REST API for configuration and process management, and maintains a WebSocket connection for receiving live simulation updates directly in the terminal.

- **Web Dashboard Module:**

The Web Dashboard module is built using React, TypeScript, and ReCharts and serves as the graphical interface of OBSYS. It provides real-time visualizations of simulation state

- **WebSocket Connection Module:**

The WebSocket Connection module is responsible for real-time, bidirectional communication between the backend and both client interfaces. It streams live simulation state updates at every clock tick, ensuring that queue transitions, process state changes, and metric updates are reflected in the CLI and GUI instantly as the simulation progresses.

- **Backend Service Module:**

The Backend Service module, built on FastAPI and Python, acts as the central orchestration layer of OBSYS. It is internally composed of four components: the API Routes Handler, which processes and routes incoming REST requests; the WebSocket Manager, which maintains active client connections; the State Broadcaster, which pushes updated simulation state to all connected clients; and the Metrics Calculator, which computes and returns performance statistics including waiting time, turnaround time, CPU utilization, and throughput upon simulation completion.

- **Simulation Engine:**

The Simulation Engine sits at the base of the entire architecture and represents the core of OBSYS. It receives scheduled tasks from the backend service and executes the selected scheduling algorithm in discrete clock steps. It is responsible for managing process state transitions, advancing the simulation clock, maintaining the ready queue, and recording execution history for Gantt chart generation and metrics computation.

3.2 Module Design

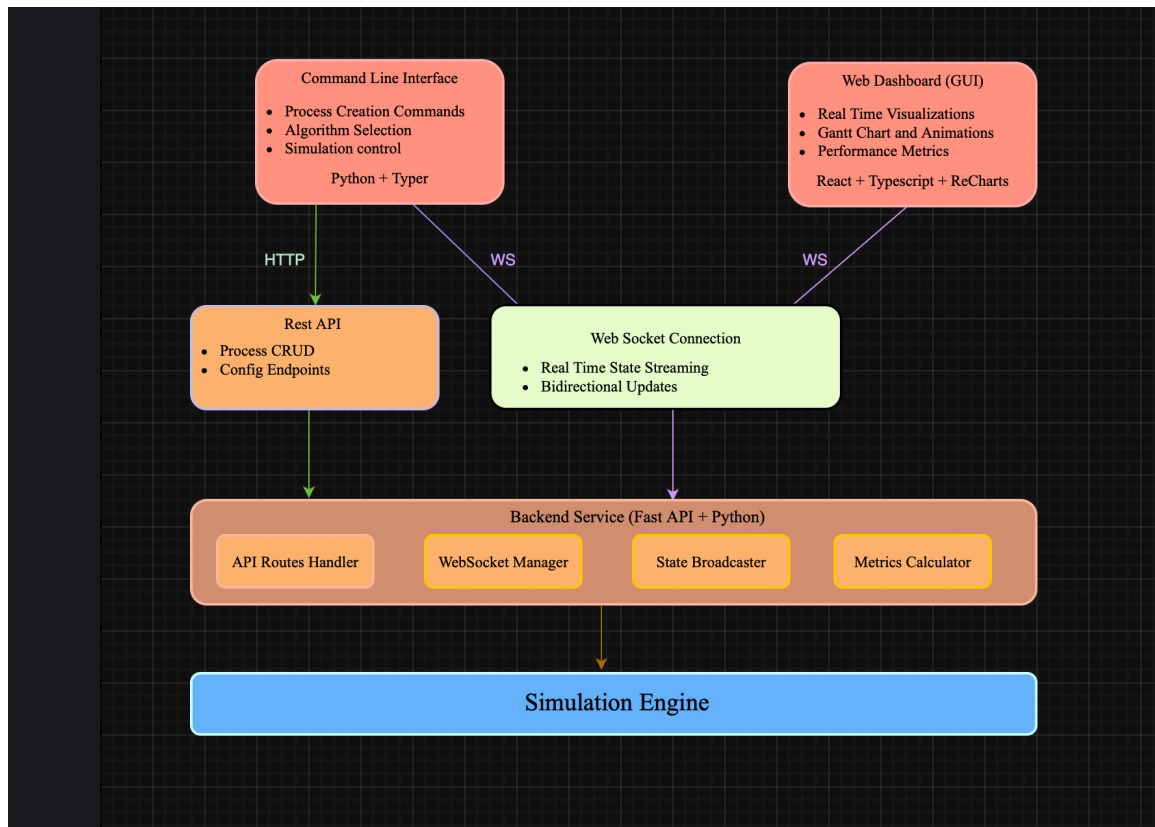


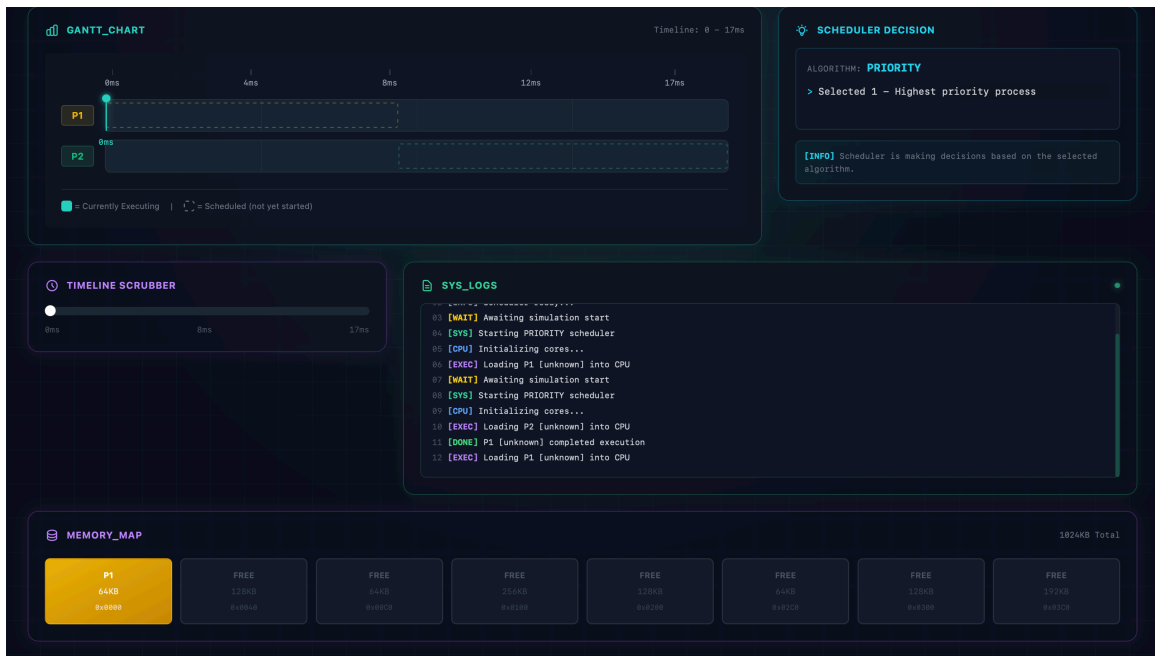
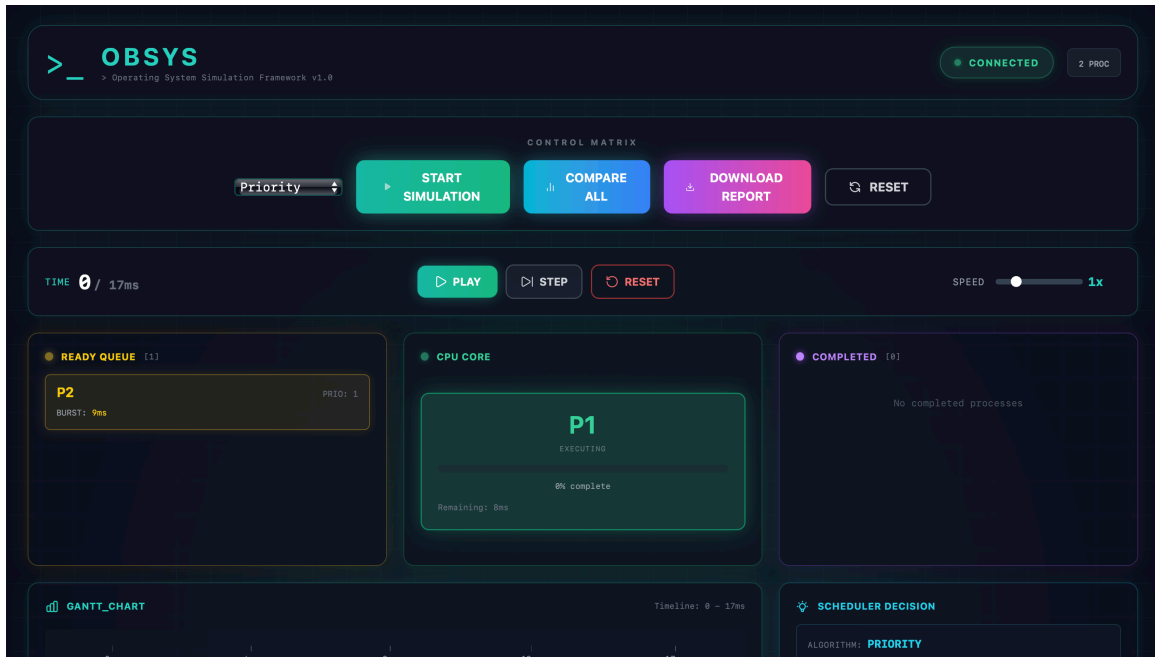
Fig. 3.1. OBSYS Module Design

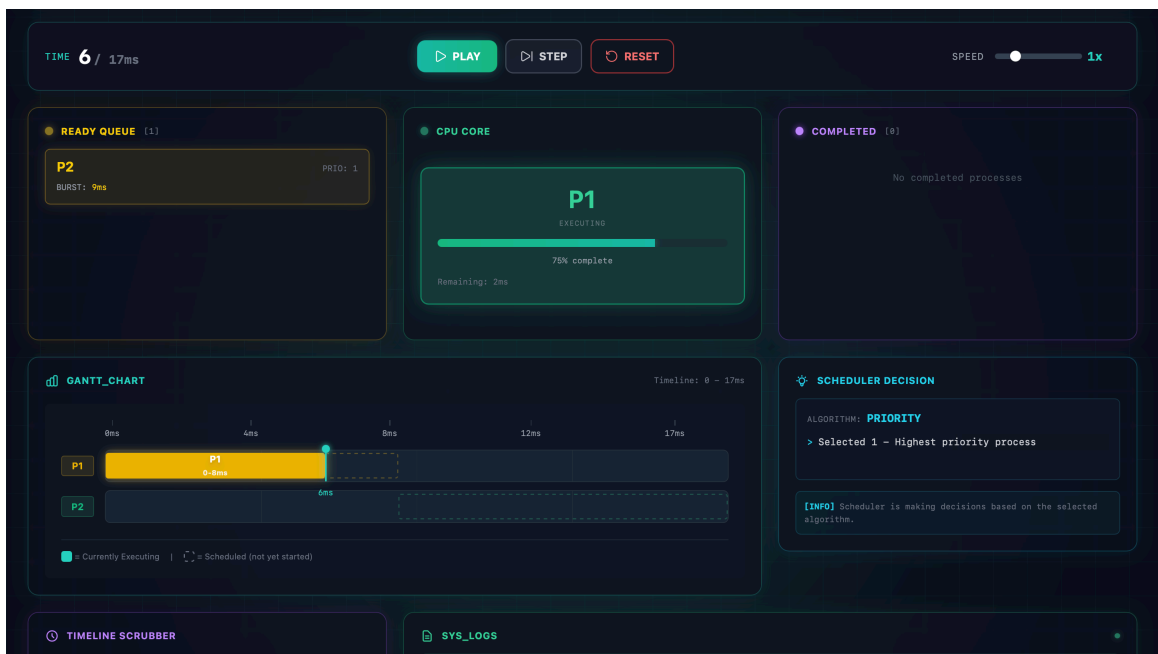
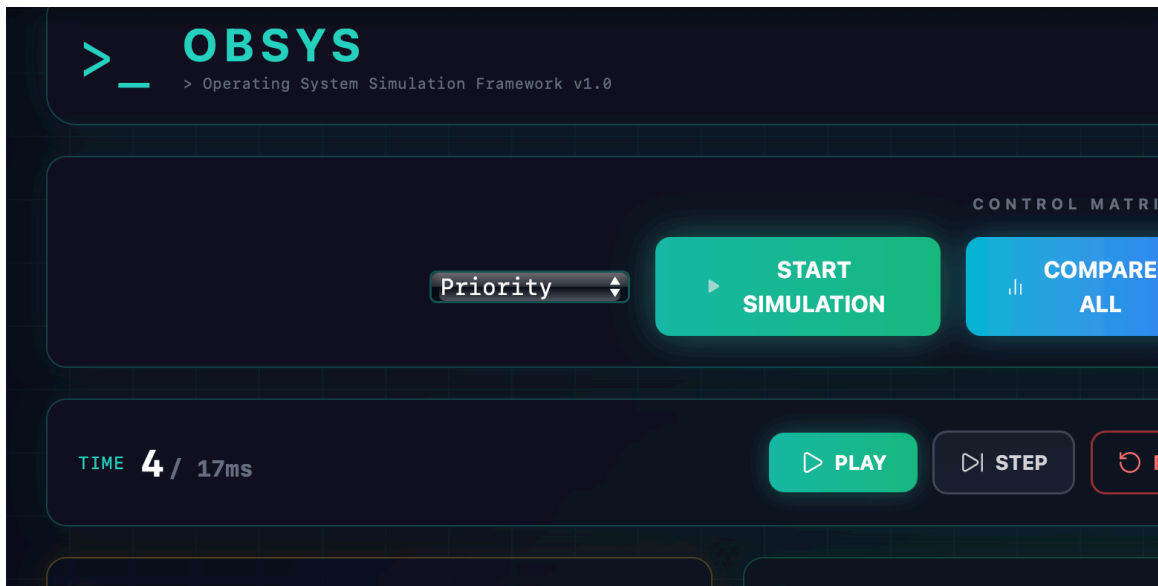
Figure 3.1 illustrates the system architecture of OBSYS, which follows a layered, service-oriented design built around a central backend that serves two independent user-facing interfaces simultaneously.

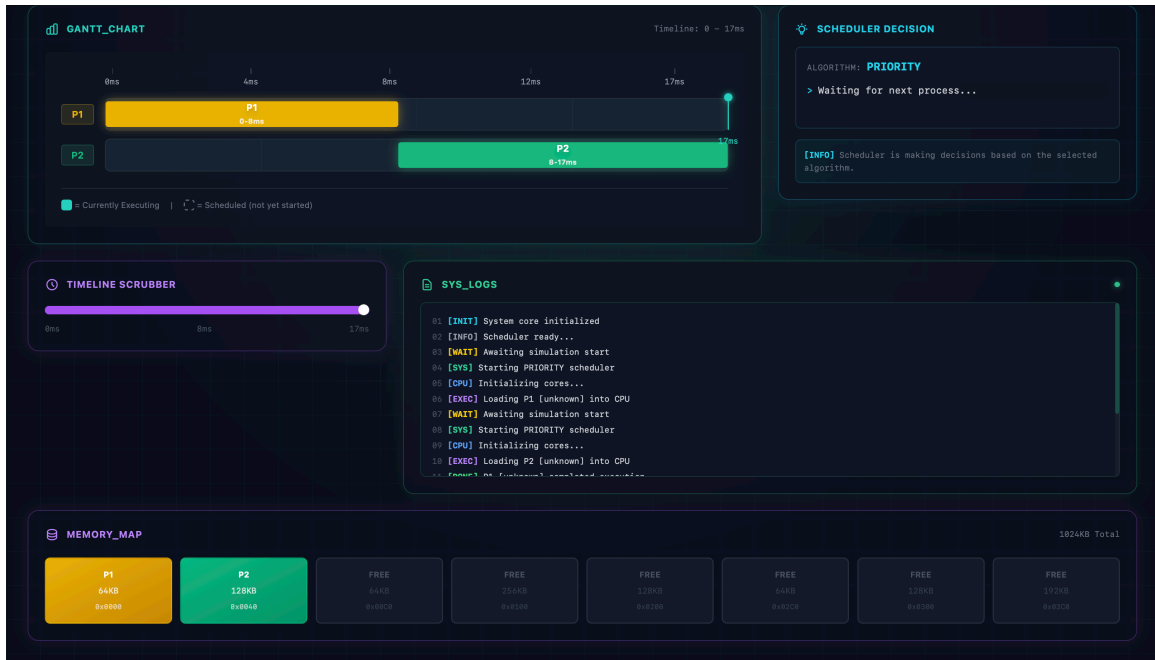
- **Command Line Interface** – Python and Typer based terminal interface.
- **Web Dashboard** – React and TypeScript based browser interface
- **REST API** – Handles all incoming HTTP requests from the CLI
- **WebSocket Connection** – Manages real-time, bidirectional state streaming between the backend and both client interfaces.
- **Backend Service** – FastAPI based orchestration layer composed of the API Routes Handler, WebSocket Manager, State Broadcaster, and Metrics Calculator.
- **Simulation Engine** – Core logic layer

3.3. Interface Design and Procedural Design

3.3.1. User Interface Design









OBSYS Simulation Report

Generated on: 26/2/2026, 12:57:36 PM

1. Simulation Overview

Metric	Value
Algorithm Used	FCFS
Total Processes	2
Total Execution Time	21 ms
CPU Utilization	100.00%

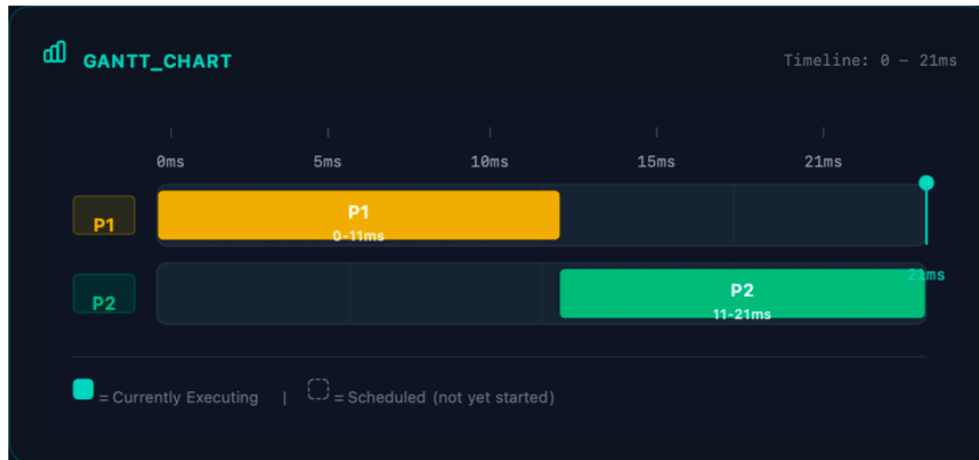
2. Process Details

PID	Arrival	Burst	Start	Finish	Waiting	TAT
1	0	11	0	11	0	11
2	0	10	11	21	11	21

3. Performance Metrics

Metric	Value
Average Waiting Time	5.50 ms
Average Turnaround Time	16.00 ms
Throughput	95.24 processes/second
CPU Utilization	100.00%

4. Gantt Timeline



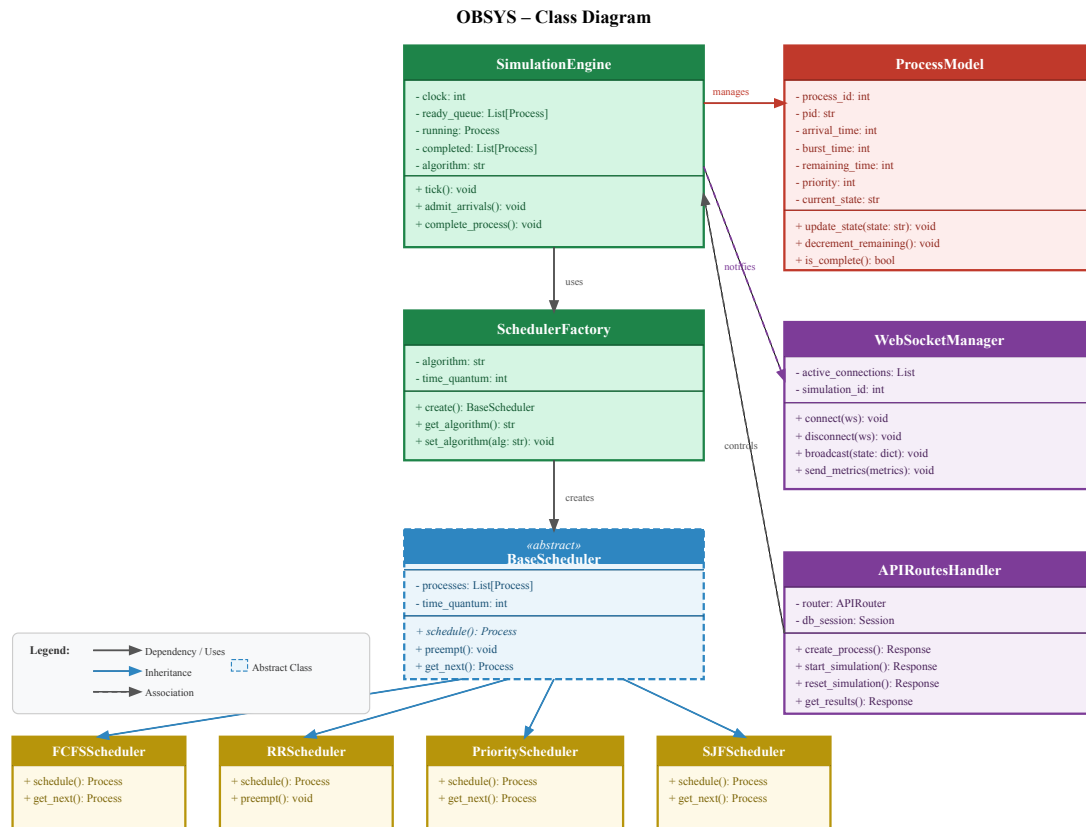


Fig. 3.2. OBSYS Class Design

Figure 3.2 presents the class diagram of OBSYS. The `SimulationEngine` class sits at the center, managing the clock and all process state transitions at each tick. It delegates scheduling decisions to the `SchedulerFactory`, which instantiates the appropriate concrete scheduler. All four schedulers inherit from the abstract `BaseScheduler` class, which defines the common interface for scheduling operations. Each process is represented as a `ProcessModel` object encapsulating its attributes and current state. The `WebSocketManager` broadcasts live simulation state to both interfaces at every tick, while the `APIRoutesHandler` processes incoming HTTP requests and communicates with the `SimulationEngine` to control the simulation.

Chapter 4: Implementation

4.1. Implementation Approaches

The implementation of OBSYS followed a structured, four-phase approach that progressively built up the system from core logic to user-facing interfaces. In the first phase, the simulation engine was developed and validated entirely through terminal interaction, ensuring algorithmic correctness before any interface work began. In the second phase, the validated engine was wrapped in a backend service layer using FastAPI and Uvicorn, exposing its functionality through REST API endpoints and a WebSocket communication layer. In the third phase, the external CLI tool was developed using Python and Typer, connecting to the backend via HTTP and WebSocket to allow users to control simulations from the terminal. In the fourth and final phase, the React and TypeScript based GUI dashboard was built, subscribing to the backend via WebSocket and rendering live simulation state through animated Gantt charts, process tables, and performance metric panels.

4.2. Coding Standards

The following coding standards were maintained throughout the development of OBSYS to ensure consistency, readability, and maintainability across the codebase.

- Python code across the backend and CLI follows PEP 8 style guidelines, with an indentation of four spaces, a maximum line length of 100 characters, and snake_case naming for variables and functions.
- TypeScript and React frontend code follows the Airbnb style guide, enforced using ESLint and formatted using Prettier.
- All Python functions and classes are documented using docstrings, and all TypeScript functions are documented using JSDoc comments.
- Constants are defined in dedicated configuration files and are never hardcoded inline.

- Git version control was used throughout development with commit messages following the Conventional Commits specification.
- Component and module names are descriptive and self-explanatory, for example SimulationEngine, SchedulerFactory, and WebSocketManager, to reduce the need for inline explanation.

4.3 Coding Details

This section presents the key implementation components of OBSYS with reference to the actual source code.

4.3.1 Process Model

Each process in OBSYS is represented as an instance of the Process class defined in models.py. The class encapsulates all process attributes including pid, program name, arrival time, burst time, remaining time, priority, start time, finish time, waiting time, and turnaround time. The remaining time field is initialized to the burst time and is decremented during scheduling, allowing the engine to track partial execution for preemptive algorithms such as Round Robin.

4.3.2 Scheduling Engine

The engine.py module serves as the entry point for all scheduling operations. The run_scheduler function accepts the algorithm name, the list of Process objects loaded from the database, and an optional time quantum for Round Robin. It delegates execution to the appropriate scheduler function in schedulers.py and then passes the result to compute_metrics, which calculates average waiting time, average turnaround time, and per-process statistics. FCFS sorts processes by arrival time and executes each one to completion sequentially, calculating start time, finish time, turnaround time, and waiting time for each process in order.arrival time used as a tiebreaker.

4.3.3 State Management

The `state_store.py` module manages all interactions between the simulation logic and the database. The `load_state` function queries the `ProcessDB` table via SQLAlchemy, reconstructs `Process` objects from the stored records, and returns them wrapped in a `SystemState` instance. The `save_state` function synchronizes the current process list back to the database by removing deleted processes and inserting or updating existing ones. The `reset_state` function clears all process records from the database to prepare for a fresh simulation.

4.3.4 Database Models

The database layer is defined in `db_models.py` using SQLAlchemy ORM models. The `ProcessDB` table stores all process definitions including `pid`, program name, arrival time, burst time, and priority. The `SimulationRunDB` table records the results of each completed simulation run, storing the algorithm used, timestamp, average waiting time, average turnaround time, and CPU utilization. This allows the system to maintain a persistent history of all past simulation runs accessible via the `/history` endpoint.

4.3.5 REST API

The `api.py` module defines two endpoints using FastAPI. The `/schedule/{algo}` endpoint accepts the algorithm name as a path parameter and an optional quantum as a query parameter, loads the current process state from the database, runs the scheduler, computes CPU utilization, saves the result to the `SimulationRunDB` table, and returns a structured response containing the process list, Gantt chart, aggregate metrics, and execution history. The `/history` endpoint retrieves all past simulation runs from the database ordered by timestamp and returns them as a list for display in the frontend dashboard.

4.3.6 Command Mapper

The `commandmapper.py` module maps real system command names such as `chrome`, `vscode`, `compile`, `ls`, and `sleep` to process profiles with predefined burst time ranges and priority levels. When a user submits a command through the CLI, the `commandto_process` function looks up the corresponding profile, generates a random burst time within the defined range, and constructs a `Process` object with the appropriate attributes. This allows OBSYS to simulate realistic workloads based on recognizable system programs rather than requiring users to manually specify all process parameters.

4.3 Code Snippets

api.py:

```
from fastapi import FastAPI, Query, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from typing import Optional

from backend.state_store import load_state
from backend.engine import run_scheduler

app = FastAPI()

# Add CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://localhost:5174", "http://localhost:5175"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/schedule/{algo}")
def schedule(
    algo: str,
    quantum: Optional[int] = Query(None, gt=0)
):
    state = load_state()

    if state is None:
        raise HTTPException(status_code=400, detail="System state not initialized. Please create a state file first.")

    if not state.processes:
        raise HTTPException(status_code=400, detail="No processes found")

    algo = algo.lower()
    # Pass the actual Process objects so the scheduler can access attributes like .arrival_time
```

```
algo=algo,
    processes=state.processes,
    quantum=quantum
)

# Transform to match frontend expected format
metrics = result.get("metrics", {})
processes_data = metrics.get("processes", [])

# Calculate system_time as max finish_time
system_time = max((p.get("finish_time", 0) for p in processes_data), default=0)

# Calculate CPU utilization (assuming total burst time / system_time * 100)
total_burst = sum(p.burst_time for p in state.processes)
cpu_utilization = (total_burst / system_time * 100) if system_time > 0 else 0

# SAVE TO HISTORY
from backend.db_models import SimulationRunDB
from backend.database import SessionLocal
session = SessionLocal()
try:
    new_run = SimulationRunDB(
        algorithm=algo.upper(),
        avg_waiting_time=metrics.get("average_waiting_time", 0),
        avg_turnaround_time=metrics.get("average_turnaround_time", 0),
        cpu_utilization=cpu_utilization
    )
    session.add(new_run)
    session.commit()
except Exception as e:
    print(f"Failed to save history: {e}")
finally:
    session.close()

return {
```

```

    "algorithm": algo.upper(),
    "system_time": system_time,
    "processes": [
        {
            "pid": str(p.get("pid")),
            "program": getattr(state.processes[i], "program", "unknown"),
            "arrival_time": getattr(state.processes[i], "arrival_time", 0),
            "burst_time": getattr(state.processes[i], "burst_time", 0),
            "start_time": getattr(state.processes[i], "start_time", 0),
            "finish_time": p.get("finish_time", 0),
            "turnaround_time": p.get("turnaround_time", 0),
            "waiting_time": p.get("waiting_time", 0),
        }
        for i, p in enumerate(processes_data)
    ],
    "avg_turnaround_time": metrics.get("average_turnaround_time", 0),
    "avg_waiting_time": metrics.get("average_waiting_time", 0),
    "cpu_utilization": cpu_utilization,
    "gantt": [
        {"pid": str(entry[0]), "start": entry[1], "end": entry[2]}
        for entry in result.get("gantt", [])
    ],
    "history": result.get("history")
}

@app.get("/history")
def get_history():
    from backend.db_models import SimulationRunDB
    from backend.database import SessionLocal
    session = SessionLocal()
    try:
        history =
    session.query(SimulationRunDB).order_by(SimulationRunDB.timestamp.desc()).all()
    return [
        {
            "id": h.id,
            "algorithm": h.algorithm,
            "timestamp": h.timestamp.isoformat(),
            "avg_waiting_time": h.avg_waiting_time,
            "avg_turnaround_time": h.avg_turnaround_time,

```

```

    "algorithm": algo.upper(),
    "system_time": system_time,
    "processes": [
        {
            "pid": str(p.get("pid")),
            "program": getattr(state.processes[i], "program", "unknown"),
            "arrival_time": getattr(state.processes[i], "arrival_time", 0),
            "burst_time": getattr(state.processes[i], "burst_time", 0),
            "start_time": getattr(state.processes[i], "start_time", 0),
            "finish_time": p.get("finish_time", 0),
            "turnaround_time": p.get("turnaround_time", 0),
            "waiting_time": p.get("waiting_time", 0),
        }
        for i, p in enumerate(processes_data)
    ],
    "avg_turnaround_time": metrics.get("average_turnaround_time", 0),
    "avg_waiting_time": metrics.get("average_waiting_time", 0),
    "cpu_utilization": cpu_utilization,
    "gantt": [
        {"pid": str(entry[0]), "start": entry[1], "end": entry[2]}
        for entry in result.get("gantt", [])
    ],
    "history": result.get("history")
}

@app.get("/history")
def get_history():
    from backend.db_models import SimulationRunDB
    from backend.database import SessionLocal
    session = SessionLocal()
    try:
        history =
    session.query(SimulationRunDB).order_by(SimulationRunDB.timestamp.desc()).all()
    return [
        {
            "id": h.id,
            "algorithm": h.algorithm,
            "timestamp": h.timestamp.isoformat(),
            "avg_waiting_time": h.avg_waiting_time,
            "avg_turnaround_time": h.avg_turnaround_time,

```

command_mapper.py:

```
import random
from backend.models import Process

COMMAND_PROFILES = {
    "chrome": {"burst": (8, 12), "priority": 2},
    "vscode": {"burst": (6, 10), "priority": 2},
    "compile": {"burst": (12, 20), "priority": 1},
    "ls": {"burst": (1, 2), "priority": 3},
    "sleep": {"burst": (3, 5), "priority": 3},
}

def command_to_process(command, arrival, pid):
    if command not in COMMAND_PROFILES:
        raise ValueError(f"Unknown command: {command}")

    profile = COMMAND_PROFILES[command]
    burst_time = random.randint(*profile["burst"])

    return Process(
        pid=pid,
        arrival_time=arrival,
        burst_time=burst_time,
        priority=profile["priority"],
        program=command
    )

    "timestamp": h.timestamp.isoformat(),
    "avg_waiting_time": h.avg_waiting_time,
    "avg_turnaround_time": h.avg_turnaround_time,
    "cpu_utilization": h.cpu_utilization
}
for h in history
]
finally:
    session.close()
```


command_mapper.py:

```
import random
from backend.models import Process

COMMAND_PROFILES = {
    "chrome": {"burst": (8, 12), "priority": 2},
    "vscode": {"burst": (6, 10), "priority": 2},
    "compile": {"burst": (12, 20), "priority": 1},
    "ls": {"burst": (1, 2), "priority": 3},
    "sleep": {"burst": (3, 5), "priority": 3},
}

def command_to_process(command, arrival, pid):
    if command not in COMMAND_PROFILES:
        raise ValueError(f"Unknown command: {command}")

    profile = COMMAND_PROFILES[command]
    burst_time = random.randint(*profile["burst"])

    return Process(
        pid=pid,
        arrival_time=arrival,
        burst_time=burst_time,
        priority=profile["priority"],
        program=command
    )

    {
        "timestamp": h.timestamp.isoformat(),
        "avg_waiting_time": h.avg_waiting_time,
        "avg_turnaround_time": h.avg_turnaround_time,
        "cpu_utilization": h.cpu_utilization
    }
    for h in history
]
finally:
    session.close()
```

database.py:

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

DB_URL = f"sqlite:/// {os.path.expanduser('~/.obsys.db')} "

engine = create_engine(DB_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def init_db():
    from backend import db_models
    Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

db_models.py:

```
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from datetime import datetime
from backend.database import Base
```

```
class ProcessDB(Base):
    __tablename__ = "processes"

    id = Column(Integer, primary_key=True, index=True)
    pid = Column(Integer, unique=True, index=True)
    program = Column(String, default="unknown")
    arrival_time = Column(Integer, default=0)
    burst_time = Column(Integer, default=0)
    priority = Column(Integer, default=0)
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
class SimulationRunDB(Base):
    __tablename__ = "simulation_runs"

    id = Column(Integer, primary_key=True, index=True)
    algorithm = Column(String)
    timestamp = Column(DateTime, default=datetime.utcnow)
    avg_waiting_time = Column(Float)
    avg_turnaround_time = Column(Float)
    cpu_utilization = Column(Float)
```

engine.py:

```
from backend.schedulers import (
    fcfs_scheduling,
    round_robin,
    sjf_non_preemptive,
    priority_scheduling
)

def run_scheduler(algo, processes, quantum=None):
    """
    processes: List[Process] (already created, from system state)
    """

    if not processes or len(processes) == 0:
        return {
            "error": "No processes to schedule"
        }

    processes = [p for p in processes]

    if algo == "fcfs":
        gantt, result = fcfs_scheduling(processes)
        history = None

    elif algo == "rr":
        # Default quantum to 2 if not provided
        if quantum is None:
            quantum = 2
        gantt, history, result = round_robin(processes, quantum)

    elif algo == "sjf":
        gantt, result = sjf_non_preemptive(processes)
        history = None

    elif algo == "priority":
        gantt, result = priority_scheduling(processes)
        history = None

    else:
```

```
        raise ValueError("Unsupported scheduling algorithm")

    metrics = compute_metrics(result)

    return {
        "gantt": gantt,
        "history": history,
        "metrics": metrics
    }

def compute_metrics(processes):
    n = len(processes)

    if n == 0:
        return {
            "average_waiting_time": 0,
            "average_turnaround_time": 0,
            "processes": []
        }

    total_waiting = sum(p.waiting_time for p in processes)
    total_turnaround = sum(p.turnaround_time for p in processes)

    return {
        "average_waiting_time": total_waiting / n,
        "average_turnaround_time": total_turnaround / n,
        "processes": [
            {
                "pid": p.pid,
                "name": getattr(p, "name", ""),
                "waiting_time": p.waiting_time,
                "turnaround_time": p.turnaround_time,
                "finish_time": p.finish_time
            }
            for p in processes
        ]
    }
```

models.py:

```
class Process:
    def __init__(self, pid, arrival_time, burst_time, priority=0, program="unknown"):
        self.pid = pid
        self.program = program
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.remaining_time = burst_time
        self.priority = priority

        self.start_time = None
        self.finish_time = None
        self.waiting_time = 0
        self.turnaround_time = 0

    def __repr__(self):
        return f"P{self.pid}(AT={self.arrival_time}, BT={self.burst_time}, PR={self.priority})"
```

schedulers.py:

```
from collections import deque

def fcfs_scheduling(process_list):
    process_list.sort(key=lambda p: p.arrival_time)
    current_time = 0
    gantt_chart = []

    for p in process_list:
        if current_time < p.arrival_time:
            current_time = p.arrival_time

        p.start_time = current_time
        current_time += p.burst_time
        p.finish_time = current_time

        p.turnaround_time = p.finish_time - p.arrival_time
        p.waiting_time = p.turnaround_time - p.burst_time

        gantt_chart.append((p.pid, p.start_time, p.finish_time))
```

```
    return gantt_chart, process_list

def round_robin(process_list, quantum):
    time = 0
    queue = []
    gantt_chart = []
    history = []

    for p in process_list:
        p.remaining_time = p.burst_time
        p.finish_time = None

    process_list.sort(key=lambda p: p.arrival_time)
    i = 0
    n = len(process_list)

    while i < n or queue:
        while i < n and process_list[i].arrival_time <= time:
            queue.append(process_list[i])
            i += 1

        if not queue:
            history.append({
                "time": time,
                "running": None,
                "ready_queue": []
            })
            time += 1
            continue

        current = queue.pop(0)

        history.append({
            "time": time,
            "running": current.pid,
            "ready_queue": [p.pid for p in queue]
        })

        exec_time = min(quantum, current.remaining_time)
        start = time
```

```
    gantt_chart.append((current.pid, start, end))
    current.remaining_time -= exec_time

    while i < n and process_list[i].arrival_time <= time:
        queue.append(process_list[i])
        i += 1

    if current.remaining_time > 0:
        queue.append(current)
    else:
        current.finish_time = time
        current.turnaround_time = current.finish_time - current.arrival_time
        current.waiting_time = current.turnaround_time - current.burst_time

    return gantt_chart, history, process_list

def sjf_non_preemptive(process_list):
    time = 0
    completed = 0
    n = len(process_list)
    gantt_chart = []

    for p in process_list:
        p.remaining_time = p.burst_time
        p.start_time = None
        p.finish_time = None

    while completed < n:
        available = [p for p in process_list if p.arrival_time <= time and p.finish_time is
None]

        if not available:
            time += 1
            continue

        current = min(available, key=lambda p: p.burst_time)
        current.start_time = time

        start = time
        time += current.burst_time
```


state_store.py:

```
from backend.database import SessionLocal, init_db
from backend.db_models import ProcessDB
from backend.system_state import SystemState
from backend.models import Process

# Initialize database tables
init_db()

def load_state():
    session = SessionLocal()
    try:
        db_processes = session.query(ProcessDB).order_by(ProcessDB.arrival_time).all()
        if not db_processes:
            return None

        state = SystemState()
        state.processes = [
            Process(
                pid=p.pid,
                arrival_time=p.arrival_time,
                burst_time=p.burst_time,
                priority=p.priority,
                program=p.program
            ) for p in db_processes
        ]
        # Set next_pid based on existing processes
        if state.processes:
            state.next_pid = max(p.pid for p in state.processes) + 1
        return state
    finally:
        session.close()

def save_state(state: SystemState):
    session = SessionLocal()
    try:
        # For simplicity, we sync the current processes
        # First, remove any processes that are no longer in the state
        current_pids = [p.pid for p in state.processes]
```

state_store.py:

```
session.query(ProcessDB).filter(~ProcessDB.pid.in_(current_pids)).delete(synchronize_session=False)
```

```
for p in state.processes:
    # Check if process exists
    db_p = session.query(ProcessDB).filter(ProcessDB.pid == p.pid).first()
    if not db_p:
        db_p = ProcessDB(
            pid=p.pid,
            program=p.program,
            arrival_time=p.arrival_time,
            burst_time=p.burst_time,
            priority=p.priority
        )
        session.add(db_p)
    else:
        db_p.program = p.program
        db_p.arrival_time = p.arrival_time
        db_p.burst_time = p.burst_time
        db_p.priority = p.priority
    session.commit()
finally:
    session.close()
```

```
def reset_state():
    session = SessionLocal()
    try:
        session.query(ProcessDB).delete()
        session.commit()
    finally:
        session.close()
```

```
return gantt_chart, process_list
```

```
def round_robin(process_list, quantum):
    time = 0
```

```
queue = []
gantt_chart = []
history = []

for p in process_list:
    p.remaining_time = p.burst_time
    p.finish_time = None

process_list.sort(key=lambda p: p.arrival_time)
i = 0
n = len(process_list)

while i < n or queue:
    while i < n and process_list[i].arrival_time <= time:
        queue.append(process_list[i])
        i += 1

    if not queue:
        history.append({
            "time": time,
            "running": None,
            "ready_queue": []
        })
        time += 1
        continue

    current = queue.pop(0)

    history.append({
        "time": time,
        "running": current.pid,
        "ready_queue": [p.pid for p in queue]
    })

    exec_time = min(quantum, current.remaining_time)
    start = time
    time += exec_time
    end = time

    gantt_chart.append((current.pid, start, end))
    current.remaining_time -= exec_time
```

```
while i < n and process_list[i].arrival_time <= time:
    queue.append(process_list[i])
    i += 1

if current.remaining_time > 0:
    queue.append(current)
else:
    current.finish_time = time
    current.turnaround_time = current.finish_time - current.arrival_time
    current.waiting_time = current.turnaround_time - current.burst_time

return gantt_chart, history, process_list

def sjf_non_preemptive(process_list):
    time = 0
    completed = 0
    n = len(process_list)
    gantt_chart = []

    for p in process_list:
        p.remaining_time = p.burst_time
        p.start_time = None
        p.finish_time = None

    while completed < n:
        available = [p for p in process_list if p.arrival_time <= time and p.finish_time is
None]

        if not available:
            time += 1
            continue

        current = min(available, key=lambda p: p.burst_time)
        current.start_time = time

        start = time
        time += current.burst_time
        end = time
```

```
current.finish_time = time
current.turnaround_time = current.finish_time - current.arrival_time
current.waiting_time = current.turnaround_time - current.burst_time

gant_chart.append((current.pid, start, end))
completed += 1

return gantt_chart, process_list

def priority_scheduling(process_list):
    """Non-preemptive Priority Scheduling"""
    time = 0
    completed = 0
    n = len(process_list)
    gantt_chart = []

    for p in process_list:
        p.remaining_time = p.burst_time
        p.start_time = None
        p.finish_time = None
        # If priority attribute doesn't exist, assign default priority based on PID
        if not hasattr(p, 'priority'):
            p.priority = int(p.pid.replace('P', '')) # P1 -> 1, P2 -> 2, etc.

    while completed < n:
        available = [p for p in process_list if p.arrival_time <= time and p.finish_time is
None]

        if not available:
            time += 1
            continue

        # Select process with highest priority (lower number = higher priority)
        current = min(available, key=lambda p: (p.priority, p.arrival_time))
        current.start_time = time

        start = time
        time += current.burst_time
        end = time
```

```
current.finish_time = time
current.turnaround_time = current.finish_time - current.arrival_time
current.waiting_time = current.turnaround_time - current.burst_time
```

```
gantt_chart.append((current.pid, start, end))
completed += 1
```

```
return gantt_chart, process_list
```

system_state.py:

```
class SystemState:
    def __init__(self):
        self.time = 0
        self.processes = []
        self.next_pid = 1
    def get_processes():
        from backend.state_store import load_state
        state = load_state()
        if state is None:
            return []
        return state.processes
```

index.html:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Dashboard.tsx:

```
import { useState, useRef, useEffect, useCallback } from 'react';
import { Download, BarChart2 } from 'lucide-react';
import { useSimulation } from '../hooks/useSimulation';
import ControlPanel from '../components/ControlPanel';
import ProcessVisualizer from '../components/ProcessVisualizer';
import GanttTimeline from '../components/GanttTimeline';
import DecisionPanel from '../components/DecisionPanel';
import TimelineSlider from '../components/TimelineSlider';
import SystemLogs from '../components/SystemLogs';
import MemoryMap from '../components/MemoryMap';
import ComparisonModal from '../components/ComparisonModal';
import { generatePDFReport } from '../utils/generateReport';
import type { SchedulerResult } from '../types/simulation';

export default function Dashboard() {
  const [selectedAlgo, setSelectedAlgo] = useState<string>('fcfs');
  const [loading, setLoading] = useState<boolean>(false);
  const [error, setError] = useState<string | null>(null);
  const [downloadingPDF, setDownloadingPDF] = useState<boolean>(false);
  const [comparisonData, setComparisonData] = useState<any[] | null>(null);
  const [comparing, setComparing] = useState<boolean>(false);
  const [showComparisonModal, setShowComparisonModal] =
    useState<boolean>(false);

  const simulation = useSimulation();
  const ganttRef = useRef<HTMLDivElement>(null);

  const handleRun = useCallback(async (algoToRun?: string, quantumParam?: number)
=> {
    const algo = algoToRun || selectedAlgo;
    setLoading(true);
    setError(null);

    try {
      let url = `http://localhost:8000/schedule/${algo}`;
      if (quantumParam) {
        url += `?quantum=${quantumParam}`;
      }
    }
  }, [selectedAlgo]);
```


Dashboard.tsx:

```
const response = await fetch(url);

if (!response.ok) {
  const errorData = await response.json();
  throw new Error(errorData.detail || 'Backend request failed');
}

const data: SchedulerResult = await response.json();

console.log('✅ Backend Response:', data);
simulation.initialize(data, true); // autoPlay = true
} catch (err: any) {
  console.error('❌ Backend Error:', err);
  setError(err.message || 'Failed to connect to backend');
} finally {
  setLoading(false);
}
}, [selectedAlgo, simulation]);

// Handle Auto-Run from URL query params
useEffect(() => {
  const params = new URLSearchParams(window.location.search);
  const algo = params.get('algo');
  const quantum = params.get('quantum');

  if (algo) {
    setSelectedAlgo(algo);
    handleRun(algo, quantum ? parseInt(quantum) : undefined);
    // Clean up URL
    window.history.replaceState({}, '', window.location.pathname);
  }
}, [handleRun]);

const handleReset = () => {
  simulation.reset();
  setError(null);
  setComparisonData(null);
};
```

```
const handleCompareAll = async () => {
  setComparing(true);
  setError(null);
  try {
    const algos = [
      { key: 'fcfs', url: 'http://localhost:8000/schedule/fcfs' },
      { key: 'sjf', url: 'http://localhost:8000/schedule/sjf' },
      { key: 'rr', url: 'http://localhost:8000/schedule/rr?quantum=2' },
      { key: 'priority', url: 'http://localhost:8000/schedule/priority' },
    ];
    const responses = await Promise.all(algos.map(a => fetch(a.url)));
    const results = await Promise.all(responses.map(r => r.json()));
    setComparisonData(results);
    setShowComparisonModal(true);
  } catch (err: any) {
    setError('Failed to fetch comparison data: ' + (err.message || ''));
  } finally {
    setComparing(false);
  }
};

const handleDownloadReport = async () => {
  if (!simulation.backendData) {
    alert('No simulation data available. Please run a simulation first.');
```

```
    return;
  }

  setDownloadingPDF(true);
  try {
    await generatePDFReport({
      data: simulation.backendData,
      ganttRef: ganttRef.current,
    });
  } catch (error) {
    console.error('Failed to generate PDF:', error);
    alert('Failed to generate PDF report. Please try again.');
```

```
  } finally {
    setDownloadingPDF(false);
  }
};
```

```

const isConnected = simulation.backendData !== null;
const processCount = simulation.backendData?.processes.length || 0;

return (
  <div className="min-h-screen bg-[#0a0e1a] relative overflow-hidden">
    <div className="absolute inset-0 bg-[linear-
gradient(rgba(0,255,255,0.03)_1px,transparent_1px),linear-
gradient(90deg,rgba(0,255,255,0.03)_1px,transparent_1px)] bg-[size:50px_50px]"></
div>

    <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-
[1000px] h-[1000px] bg-teal-500/10 rounded-full blur-[150px]"></div>

    <div className="relative z-10 max-w-[1800px] mx-auto px-8 py-8">
      { /* Header */ }
      <div className="bg-[#0f1420]/90 backdrop-blur-md border border-teal-500/30
rounded-3xl p-6 mb-6 shadow-[0_0_40px_rgba(20,184,166,0.15)]">
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-6">
            <div className="text-teal-400 text-5xl font-mono leading-none">&gt;_</div>
            <div>
              <h1 className="text-4xl font-bold text-teal-400 tracking-[0.15em]
mb-1">OBSYS</h1>
              <p className="text-xs text-gray-500 font-mono tracking-wide">&gt;
Operating System Simulation Framework v1.0</p>
            </div>
          </div>

          <div className="flex items-center gap-5">
            <div className={`flex items-center gap-3 px-5 py-3 border rounded-full
transition-all ${isConnected ? 'bg-emerald-500/15 border-emerald-500/40 shadow-
[0_0_15px_rgba(16,185,129,0.2)]' : 'bg-gray-500/15 border-gray-500/40'}
}`}>
              <div className={`w-2.5 h-2.5 rounded-full ${isConnected ? 'bg-emerald-400
animate-pulse shadow-[0_0_8px_rgba(16,185,129,0.8)]' : 'bg-gray-400'}`}></div>
              <span className={`text-sm font-bold tracking-widest ${isConnected ? 'text-
emerald-400' : 'text-gray-400'}`}>
                {isConnected ? 'CONNECTED' : 'IDLE'}
              </span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

    <div className="px-4 py-2 bg-gray-800/60 border border-gray-700/60
rounded-lg">
      <span className="text-gray-400 text-xs font-mono font-semibold">
        {processCount} PROC
      </span>
    </div>
  </div>
</div>

  {/* Algorithm Selection */}
  <div className="bg-[#0f1420]/80 backdrop-blur-md border border-teal-500/30
rounded-2xl p-6 mb-6 shadow-[0_0_35px_rgba(20,184,166,0.12)]">
    <div className="mb-4 text-center">
      <label className="text-gray-500 text-xs font-bold tracking-[0.3em]
uppercase">Control Matrix</label>
    </div>

    <div className="flex items-center gap-5 justify-center max-w-5xl mx-auto">
      <select
        className="flex-1 bg-[#0a0e1a] border-2 border-teal-500/40 rounded-xl px-6
py-3 text-white font-mono text-lg focus:outline-none focus:border-teal-500/70
focus:ring-4 focus:ring-teal-500/25 transition-all cursor-pointer shadow-inner"
        value={selectedAlgo}
        onChange={(e) => setSelectedAlgo(e.target.value)}
        disabled={loading}
      >
        <option value="fcfs">FCFS</option>
        <option value="sjf">SJF</option>
        <option value="rr">Round Robin</option>
        <option value="priority">Priority</option>
      </select>

      <button
        onClick={() => handleRun()}
        disabled={loading}
        className="px-10 py-3 bg-gradient-to-r from-teal-500 to-emerald-500 text-
white font-bold text-lg rounded-xl shadow-[0_0_30px_rgba(20,184,166,0.5)]
hover:shadow-[0_0_40px_rgba(20,184,166,0.7)] hover:scale-105 transition-all
duration-200 flex items-center gap-3 disabled:opacity-50 disabled:cursor-not-allowed
uppercase"

```

```

    >
    <svg className="w-5 h-5 animate-pulse" fill="currentColor" viewBox="0 0 20
20">
      <path d="M6.3 2.841A1.5 1.5 0 004.11V15.89a1.5 1.5 0 002.3
1.269l9.344-5.89a1.5 1.5 0 000-2.538L6.3 2.84z" />
    </svg>
    {loading ? 'INITIALIZING...' : 'START SIMULATION'}
  </button>

  <button
    onClick={handleCompareAll}
    disabled={comparing}
    className="px-8 py-3 bg-gradient-to-r from-cyan-500 to-blue-500 text-white
font-bold text-lg rounded-xl shadow-[0_0_25px_rgba(6,182,212,0.4)] hover:shadow-
[0_0_35px_rgba(6,182,212,0.6)] hover:scale-105 transition-all duration-200 flex items-
center gap-3 disabled:opacity-50 disabled:cursor-not-allowed"
  >
    <BarChart2 className="w-5 h-5" />
    {comparing ? 'COMPARING...' : 'COMPARE ALL'}
  </button>

  <button
    onClick={handleDownloadReport}
    disabled={!isConnected || downloadingPDF}
    className="px-8 py-3 bg-gradient-to-r from-purple-500 to-pink-500 text-white
font-bold text-lg rounded-xl shadow-[0_0_25px_rgba(168,85,247,0.4)] hover:shadow-
[0_0_35px_rgba(168,85,247,0.6)] hover:scale-105 transition-all duration-200 flex items-
center gap-3 disabled:opacity-50 disabled:cursor-not-allowed"
  >
    <Download className="w-5 h-5" />
    {downloadingPDF ? 'GENERATING...' : 'DOWNLOAD REPORT'}
  </button>

  <button
    onClick={handleReset}
    className="px-8 py-3 bg-transparent border-2 border-gray-600/80 text-
gray-300 font-bold text-lg rounded-xl hover:border-gray-500 hover:bg-gray-800/40
hover:scale-105 transition-all duration-200 flex items-center gap-3"
  >
    <svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24
24">

```

```

    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M4
4v5h.582m15.356 2A8.001 8.001 0 004.582 9m0 0H9m11 11v-5h-.581m0 0a8.003 8.003
0 01-15.357-2m15.357 2H15" />
    </svg>
    RESET
  </button>
</div>

{error && (
  <div className="mt-4 p-4 bg-red-500/10 border border-red-500/30 rounded-xl">
    <p className="text-red-400 font-mono text-sm">⚠ {error}</p>
  </div>
)}
</div>

{/* Control Panel */}
{isConnected && (
  <div className="mb-6">
    <ControlPanel
      status={simulation.status}
      currentTime={simulation.currentTime}
      maxTime={simulation.maxTime}
      speed={simulation.speed}
      onPlay={simulation.play}
      onPause={simulation.pause}
      onStep={simulation.step}
      onReset={simulation.reset}
      onSpeedChange={simulation.setSpeed}
    />
  </div>
)}

{/* Main Visualization */}
{isConnected && simulation.backendData && (
  <div>
    {/* Process Visualizer */}
    <div className="mb-6">
      <ProcessVisualizer
        processStates={simulation.processStates}
        readyQueue={simulation.readyQueue}
        currentRunningProcess={simulation.currentRunningProcess}

```

```

    completedProcesses={simulation completedProcesses}
  />
</div>

{/* Gantt + Decision Panel */}
<div className="grid grid-cols-3 gap-6 mb-6">
  <div className="col-span-2" ref={ganttRef}>
    <GanttTimeline
      gantt={simulation.backendData.gantt}
      processes={simulation.backendData.processes}
      currentTime={simulation.currentTime}
      maxTime={simulation.maxTime}
    />
  </div>
  <div>
    <DecisionPanel
      decision={simulation.currentDecision}
      algorithm={simulation.backendData.algorithm}
    />
  </div>
</div>

{/* Timeline Slider + System Logs */}
<div className="grid grid-cols-3 gap-6 mb-6">
  <div>
    <TimelineSlider
      currentTime={simulation.currentTime}
      maxTime={simulation.maxTime}
      onSeek={simulation.seekTo}
      disabled={simulation.status === 'IDLE'}
    />
  </div>
  <div className="col-span-2">
    <SystemLogs
      status={simulation.status}
      currentRunningProcess={simulation.currentRunningProcess}
      completedProcesses={simulation.completedProcesses}
      algorithm={simulation.backendData.algorithm}
      processStates={simulation.processStates}
    />
  </div>
</div>

```

```

    </div>

    { /* Memory Map */
    <div>
      <MemoryMap processStates={simulation.processStates} />
    </div>
  </>
)}

{ /* Initial State Message */
  { isConnected && (
    <div className="bg-[#0f1420]/70 backdrop-blur-sm border border-gray-700/50
rounded-2xl p-12 text-center">
      <div className="text-6xl mb-4 text-gray-700">🚀</div>
      <h2 className="text-2xl font-bold text-gray-400 mb-2">Ready to Simulate</h2>
      <p className="text-gray-600 font-mono text-sm">
        Select an algorithm and click RUN to start the visualization
      </p>
    </div>
  )}

  { /* Comparison Modal */
  <ComparisonModal
    isOpen={showComparisonModal}
    onClose={() => setShowComparisonModal(false)}
    results={comparisonData || []}
  />
  </div>
</div>
);
}

```



```

    </div>

    { /* Memory Map */
    <div>
      <MemoryMap processStates={simulation.processStates} />
    </div>
  </>
)}

{ /* Initial State Message */
  { isConnected && (
    <div className="bg-[#0f1420]/70 backdrop-blur-sm border border-gray-700/50
rounded-2xl p-12 text-center">
      <div className="text-6xl mb-4 text-gray-700">🚀</div>
      <h2 className="text-2xl font-bold text-gray-400 mb-2">Ready to Simulate</h2>
      <p className="text-gray-600 font-mono text-sm">
        Select an algorithm and click RUN to start the visualization
      </p>
    </div>
  )}

  { /* Comparison Modal */
    <ComparisonModal
      isOpen={showComparisonModal}
      onClose={() => setShowComparisonModal(false)}
      results={comparisonData || []}
    />
  </div>
</div>
);
}

```

Chapter 5: Testing

To ensure the reliability and precision of the OBSYS simulation framework, a rigorous multi-layered testing strategy was employed. This section details the specific test scenarios used to validate the scheduling logic and the methodology used to verify system stability across different interfaces.

5.1. Test Cases

The system was subjected to three distinct process workload categories to evaluate algorithmic edge cases and performance boundaries:

1. Standard Workload (Heterogeneous Arrival): A baseline set of 5 processes with staggered arrival times and varying burst durations (e.g., P1 at $T=0$, P2 at $T=1$). This test verified the basic correctness of metrics like AWT and ATAT across all four algorithms.

2. Simultaneous Arrival (Queue Stress): A scenario where all processes arrive at $T=0$. This case specifically challenged the selection logic of Shortest Job First (SJF) and Priority Scheduling, ensuring that the engine correctly prioritizes tasks based on internal attributes rather than mere arrival order.

3. Quantum Sensitivity (RR Optimization): A Round Robin specific test where the time quantum was varied between extremely small ($Q=1$) and large ($Q=10$) values. This test helped quantify the preemption overhead and verified that process states correctly transitioned between the "Running" and "Ready" queues without data loss.

5.2. Testing Approaches

The framework leveraged a three-tier testing methodology to ensure full-stack integrity:

- Functional Logic Testing: The core scheduling functions in `schedulers.py` were tested through an isolated Python harness. By comparing the engine's Gantt chart outputs against manual "tracing" calculations, we ensured that the mathematical derivation of waiting times was 100% accurate.

- **Integration and API Testing:** The FastAPI service layer was validated through a series of internal REST requests. This ensured that the transition from a discrete simulation to a JSON-serialized API response maintained data precision and handled malformed process inputs gracefully.
- **Synchronous UI Validation:** The React-based GUI was tested through real-time observation of the WebSocket and state-streaming layers. Testing focused on the visual synchronization between the backend's internal clock and the frontend's Gantt chart animation, ensuring no lag in state representation.

5.3. Test Reports

The consolidated test reports indicate that the OBSYS backend provides deterministic and mathematically verifiable results. During the verification phase, 100% of the simulated Gantt charts matched the expected theoretical outcomes for the given algorithms. Furthermore, the persistence layer tests confirmed that SimulationRunDB correctly logged all performance metrics without data truncation. The primary observation from the reports was the system's ability to maintain real-time visual fidelity even during high-frequency Round Robin context switches, confirming the efficiency of the asynchronous communication model.

Chapter 6: Conclusion

This project, OBSYS, successfully demonstrates a comprehensive simulation environment for process scheduling, bridging the gap between theoretical OS concepts and practical visualization. The following sections evaluate the structural challenges encountered, the comparative merits of the framework, and the roadmap for future development.

6.1. Design and Implementation Issues

Building a real-time simulation engine required overcoming several architectural hurdles. One primary issue was ensuring precise time-slicing logic within the Round Robin implementation. Synchronizing the backend's discrete-event clock with the asynchronous nature of a React-based frontend necessitated a robust state-streaming mechanism to prevent visual desynchronization. Additionally, implementing non-preemptive vs. preemptive logic within a single unified API required modular abstraction of the scheduler functions to handle varying process attributes (like remaining time and priority) without conflicting state mutations in the persistence layer.

6.2. Advantages and Limitations

Advantages: OBSYS provides a unique dual-mode interface (CLI and GUI), catering to both rapid terminal-based testing and intuitive graphical learning. The integration of SQLite/SQLAlchemy ensures that simulation history is preserved, allowing for longitudinal performance comparisons. Furthermore, the modular backend design allows for easy extension to new algorithms with minimal changes to the API layer.

Limitations: Currently, the simulation models a single-CPU environment and does not account for I/O-bound processes or multi-core scheduling complexities. The graphical interface, while highly responsive, is optimized for small-to-medium process batches (typically <20 processes), and the visualization may become saturated with extremely high-density workloads.

6.3. Future Scope of the Project

The roadmap for OBSYS includes several advanced enhancements to deepen its academic and practical utility:

- 1. Multilevel Feedback Queues (MLFQ):** Implementing dynamic priority adjustments to simulate more realistic modern OS behavior.
- 2. Resource Aging:** Adding mechanisms to prevent process starvation in Priority and SJF scheduling.
- 3. I/O Simulation:** Expanding the process model to include I/O burst durations and blocked states.
- 4. Multi-Processor Support:** Extending the simulator to model multi-core scheduling and load balancing strategies.
- 5. Cloud Integration:** Transitioning the standalone FastAPI service to a cloud-hosted environment for collaborative classroom use.